

# Aufgabe 2: Choreograph

Team-ID: 01004

Team: Wawa

Bearbeiter dieser Aufgabe:  
Raphael Porsche

26. August 2025

## Inhaltsverzeichnis

|                          |    |
|--------------------------|----|
| Programm-Ausführung..... | 1  |
| Lösungsidee.....         | 2  |
| Umsetzung.....           | 5  |
| Werkzeuge.....           | 8  |
| Packages:.....           | 8  |
| Beispiele.....           | 8  |
| Quellcode.....           | 12 |

## Programm-Ausführung

Programmiersprache: Odin Version: dev-2025-02:584fdc0d4

Getestet auf: Linux (ZorinOS 17)

Packages: core:fmt, core:unicode/utf8, core:os, core:strings, core:sort, core:math, core:path/filepath, vendor:raylib

Benutzungsanweisung:

1. Ein Terminal öffnen.
2. Arbeitsverzeichnis auf das „Aufgabe2\_choreograph“ Verzeichnis bewegen.
3. Diesen Befehl ausführen: „./choreograph.bin“.

Selbstkompilierung:

1. Odin mit den Anweisungen auf <https://odin-lang.org/docs/install/> installieren.
2. Ein Terminal öffnen, das Arbeitsverzeichnis auf das „Aufgabe2\_choreograph“ Verzeichnis
3. Diesen Befehl ausführen: „odin build choreograph“.
4. Nun sollte die ausführbare Datei im „Aufgabe2\_choreograph“ Verzeichnis durch eine neue ersetzt sein.

## Lösungsidee

Um alle möglichen Kombinationen von Figuren zu finden, die eine passende Choreographie bilden, ist ein Brute-Force-Ansatz ein einfacher Weg. Allerdings lässt sich der Suchbereich leicht weiter einschränken, wodurch wir die Laufzeit minimieren können. Hierfür verwenden wir diese Methoden:

1. Jede Figur kann nur begrenzt häufig in einer Choreographie vorkommen, da die Gesamtlänge der Choreographie festgelegt ist. Eine Figur kann in einer Choreographie in jedem Fall nur so häufig auftauchen, bis ihre summierte Länge die Gesamtlänge der Choreographie überschreitet. So können wir für jede Figur ein grobes Intervall erstellen, wie häufig sie auftreten darf.

Die möglichen Häufigkeiten einer Figur sind somit alle ganzen Zahlen im geschlossenen Intervall  $[0, x]$ , wobei:

**Gleichung 1:**  $x = \lfloor g_l / f_l \rfloor$

| Symbol | Bedeutung                            |
|--------|--------------------------------------|
| $g_l$  | Die gesamte Länge der Choreographie. |
| $f_l$  | Die Länge der betroffenen Figur.     |

2. Wir suchen zuerst nach allen Kombinationen von Figuren, die exakt die gewünschte Gesamtlänge ergeben. Dabei ignorieren wir zunächst das tatsächliche Anwenden der Figuren auf die Positionen der Tänzer.

Hier ist die Reihenfolge der Kombination irrelevant und dadurch die Suche schneller und effizienter. Wir verwenden eine **Tiefensuche**, die alle mögliche Kombinationen von Figurenhäufigkeiten (die vorher in **Methode 1** eingeschränkt wurden) durchläuft.

In der Tiefensuche erstellen wir einen Suchbaum. Jede Ebene des Baumes stellt eine Figur dar. Die Knoten dieser Ebene repräsentieren dann immer genau eine Häufigkeit dieser Figur. Zum Beispiel: in Ebene 1 betrachten wir Figur X und erstellen 4 Knoten: 0 mal Figur X, 1 mal Figur X, 2 mal Figur X, 3 mal Figur X.

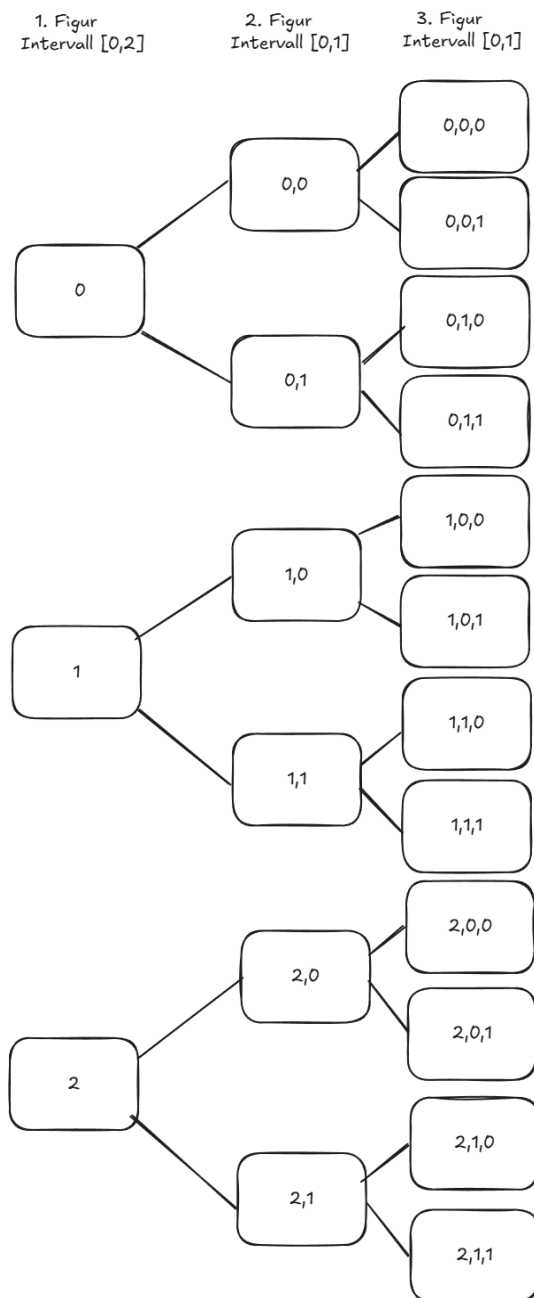


Abbildung 1

Ausgehend von einem solchen Knoten erzeugen wir für die nächste Figur neue Knoten, indem wir dieser Figur jede ihrer möglichen Häufigkeiten als neuen Knoten zuweisen.

Auf diese Weise wächst der Suchbaum: Auf der ersten Ebene stehen alle möglichen Häufigkeiten der ersten Figur, auf der zweiten alle Kombinationen der ersten beiden Figuren usw. Da wir schlussendlich nur die letzte Ebene benötigen, löschen wir Knoten, dessen Kinder wir schon ermittelt haben.

Wenn die Gesamtlänge einer dieser unfertigen Choreographien zu irgendeinem Zeitpunkt die Gesamtlänge der kompletten Choreographie übersteigt, wird sie verworfen, da sie in keinem Fall zur richtigen Zeitmenge zurückkehren kann. Nachdem wir durch alle Figuren durchgegangen sind, eliminieren wir alle weiteren Choreographien, die nicht die gewünschte Gesamtlänge haben.

Wir erhalten so alle mögliche Häufigkeitsverteilungen der Figuren, für eine passende Choreographie, wodurch wir anschließend die richtigen Reihenfolgen in einem viel kleineren Suchbereich überprüfen können.

3. Wenn wir uns die Laufzeit der Tiefensuche aus **Methode 2** genauer anschauen, kommen wir leicht auf einen weiteren Weg sie zu optimieren. Als Beispiel verwenden wir diese Figuren und ignorieren die eigentliche Permutation der Positionen der Tänzer.

| Figurname | Maximales Vorkommen (Vektor a) | Index |
|-----------|--------------------------------|-------|
| a_1       | 5                              | 0     |
| a_2       | 2                              | 1     |

|     |   |   |
|-----|---|---|
| a_3 | 3 | 2 |
|-----|---|---|

Wenn wir den Regeln aus **Methode 2** befolgen, benötigt die erste Ebene des Suchbaums eine Laufzeit von 6 Schritten (0 bis 5 mal darf diese Figur auftauchen). Wir erstellen Knoten für alle möglichen Vorkommen der Figur. Die nächste Ebene des Suchbaums hätte dann eine Laufzeit von  $6 * 3$  Schritten, da für jede Häufigkeit von a\_1 jede Häufigkeit aus a\_2 durchgegangen wird und ein neuer Knoten erstellt. Die dritte Ebene hätte hier dann eine Laufzeit von  $5 * 3 * 4$  Schritten. Dieses Verhalten lässt sich mit dieser Formel generalisieren:

$$O(i_{max}) = \sum_{i=1}^{i_{max}} \prod_{j=1}^i a_j$$

| Symbol | Bedeutung                                                                                                |
|--------|----------------------------------------------------------------------------------------------------------|
| a_j    | Der Vektor von den maximalen Häufigkeiten aller Figuren, indexiert an j (erstes Element an der Stelle 1) |
| i_max  | Die Länge von a.                                                                                         |

Die gesamte Laufzeit in unserem Beispiel ist so  $6 + 6 * 3 + 6 * 3 * 4 = 96$  Schritte. In dieser Formel fällt auf, dass wenn wir die Reihenfolge der Aktionen anders anordnen, auch die Laufzeit sich verändert. Wenn die Figuren nach dem möglichst kleinsten maximalen Vorkommen vorsortiert werden, ist die Laufzeit am kürzesten:  $3 + 3 * 4 + 3 * 4 * 6 = 87$  Schritte.

Der Fakt, dass manche unfertige Choreographien vorher abgebrochen werden, da sie die gewünschte Gesamtlänge überschreiten, macht diesen Ansatz noch hilfreicher. Figuren mit kleinem maximalen Vorkommen haben eine große Länge, wodurch eine große Menge von unfertigen Choreographien schon früh die Gesamtlänge überschreiten und nicht weiter durchsucht werden müssen.

4. Für jede Häufigkeitsverteilung aus **Methode 2** suchen wir nach allen möglichen Anordnungen der Figuren, die die Tänzer wieder zu ihrer Ausgangsposition zurückbringt. Wenn wir eine finden würden, wäre sie in jedem Fall eine passende Choreographie, da sie die Tänzer zu ihrer Ausgangsposition zurückbringt und garantiert die richtige Länge hat.

Wir prüfen rekursiv jede mögliche Permutation der Figuren einer Häufigkeitsverteilung. Dafür durchsuchen wir jede mögliche Figur die zu einem Zeitpunkt ausgeführt werden kann

und erstellen so eine Art Suchbaum. Jeder Knoten des Suchbaums speichert eine eigene Häufigkeitsverteilung, die momentane Position der Tänzer und welche Figuren bis zu diesem Punkt durchgeführt worden.

Beim rekursiven Traversieren erstellen wir für jeden Knoten so viele weitere Knoten in der Ebene darunter, wie Figuren in ihrer Häufigkeitsverteilung noch durchzuführen sind. Die ausgesuchte Figur für einen neuen Knoten wird an die vorherigen Positionen der Tänzer angewendet und ihre neue Häufigkeitsverteilung wurde bei der ausgewählten Figur um eins vermindert.

Wenn die Häufigkeitsverteilung zu einem bestimmten Zeitpunkt nur noch für jede Figur „0“ ist, also alle benötigten Figuren ausgeführt, dann überprüfen wir ob die Tänzer wieder zu ihrer Startposition zurückgekehrt sind. Wenn das der Fall ist haben wir eine passende Choreographie gefunden und wir geben alle bis dorthin ausgeführten Figuren in der richtigen Reihenfolge zurück.

Zum Schluss müssen wir nur alle möglichen Choreographien nach den aufgabenbezogenen Kriterien betrachten und uns jeweils die passendste aussuchen.

Für das erste Kriterium: „Möglichst viele unterschiedliche Figuren werden eingebaut“, gehen wir durch jede Choreographie durch und überprüfen wie viele von allen Figuren dort jeweils auftauchen. Wir geben dann die best passendste Choreographie zurück.

Für das zweite und dritte Kriterium: „Möglichst viele Figuren werden eingebaut“ und „Möglichst wenige Figuren werden eingebaut.“ schauen wir uns einfach nur die Längen ausgeführten Figuren an. Wir geben die best und schlechtesten passende Choreographie zurück.

Für das vierte und fünfte Kriterium: „Die von allen Tanzenden insgesamt zurückgelegte Strecke soll möglichst groß sein.“ und „Die von allen Tanzenden insgesamt zurückgelegte Strecke soll möglichst klein sein.“ berechnen wir zuerst die zurückgelegte Strecke aller möglichen Figuren. Hierfür summieren wir einfach nur die Differenzen zwischen die Ausgangsposition und Endposition aller Tänzer in einer Figur. Die zurückgelegte Strecke einer ganzen Choreographie ist dann die Summe der zurückgelegten Strecken aller vorhandenen Figuren. Schlussendlich geben wir dann die best und schlechtesten passende Choreographie zurück.

## Umsetzung

(Q1 – Q5 referenziert verschiedene Teile des Quellcodes)

Bei der Umsetzung unserer Problemlösung erstellen wir zuerst eine passende Struktur „Action“, die alle wichtige Information einer Figur speichert:

```
Configuration :: [16]int
base_config : Configuration : {0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15}

Action :: struct {
    name : string,
```

```
    length : int,  
    reconfig : Configuration  
}
```

Eine Figur speichert ihren Namen für die Auswertung am Ende des Programms und ihre Länge, um jederzeit zu Wissen, wie viel Zeit einer vollen Choreographie sie einnimmt. Außerdem speichern wir ihre „reconfig“, also an welcher Stelle die Tänzer nach einem Anwenden der Figur auf die Ausgangsposition stehen würden. (Q1)

Wir füllen solch eine Struktur für jede eingegebene Figur aus und fassen sie in einem Vektor zusammen. Wir sortieren diesen Vektor nach ihrer Länge. **Gleichung 1** zeigt, dass die Länge einer Figur antiproportional zu ihrer maximalen Häufigkeit ist. Figuren mit größere Länge sollten so weiter vorne stehen, damit wir den Effekt aus **Methode 3** erreichen. Anschließend ermitteln wir die einzelnen maximalen Häufigkeiten für jede Figur in einem weiteren Vektor mit einer einfachen Implementation von **Methode 1**. (Q2)

Im nächsten Schritt befolgen wir **Methode 2** um an alle möglichen Kombinationen von Figurenhäufigkeiten zu kommen. Wir erstellen einen Vektor von Vektoren von Zahlen. Jedes Element in diesem Vektor stellt einen Knoten des Suchbaumes dar, in dem wir, wie **Abbildung 1** zeigt, die Häufigkeiten aller bisherigen ausgesuchten Figuren speichern. Wir erstellen alle möglichen Knoten der ersten Ebene in unserem Vektor und starten den Kreislauf der Tiefensuche. Hierbei behandeln wir in jeder Iteration das erste Element in dem Vektor. Wie auch **Abbildung 1** zeigt, erstellen wir für den jetzigen Knoten weitere Kinder in der Ebene darunter mit der Ausnahme von diesen Fällen (Q3) :

1. Die momentane Gesamtlänge der unfertigen Choreographie überschreitet die gewünschte Gesamtlänge der Choreographie. Hier erstellen wir keine Kinder, da sie in keinem Fall die gewünschte Gesamtlänge erreichen können.
2. Wir sind bei der letzten Figuren-ebene angekommen und es gibt keine weiteren Figuren. Wir wissen die Figuren-ebene eines Element in dem Vektor, indem wir einfach seine Länge überprüfen. Hier prüfen wir außerdem ob die Länge dieser fertigen Choreographie genau die gewünschte Länge ergibt. Wenn das der Fall ist, haben wir eine mögliche Häufigkeitsverteilung gefunden und speichern sie ab.

Nachdem wir einen Knoten behandelt haben, löschen wir ihn aus dem Vektor. Anstatt alle Elemente dabei einen vorzurücken, tauschen wir das erste und letzte Element und löschen erst dann das neue letzte Element. Bei diesem Vorgang ist die Laufzeit konstant, und wächst nicht linear, als wenn wir alle Elemente einen vorrücken würden. Hierdurch haben wir auch eine Tiefensuche anstatt einer Breitensuche, da wir immer das letzte Kind des jetzigen Knoten vor allen anderen Elementen in der gleichen Ebene behandeln. Wir wiederholen diesen Vorgang solange, bis der Vektor keine Elemente mehr hat. (Q3)

Als nächstes implementieren wir **Methode 4**. Wir erstellen eine rekursive Funktion, die ähnlich wie bei **Methode 2**, eine Art Suchbaum für jede Häufigkeitsverteilung durchsucht. Hierbei ist jeder „Knoten“ des Suchbaums eine weitere Instanz der rekursiven Funktion. Wir geben bei der Erstellung dieses Suchbaums die verschiedenen Figuren, die noch ausgeführt werden müssen,

welche Figuren in welcher Reihenfolge schon ausgeführt wurden und an welcher Stelle sich die Tänzer befinden, weiter. (Q4)

Für jeder Instanz der rekursiven Funktion, gehen wir folgende Schritte durch:

1. Wir überprüfen ob die übrigbleibende Häufigkeitsverteilung nur noch ein Vektor von „0“ ist. Ist das der Fall, dann vergleichen wir die jetzige Position der Tänzer mit ihrer Ausgangsposition. Ist sie gleich, dann haben wir eine passende Choreographie gefunden und geben sie als eine passende Variante zurück. Ist sie ungleich, dann löschen wir den jetzigen Knoten.
2. Wir gehen die übrigbleibende Häufigkeitsverteilung der verschiedenen Figuren durch und erstellen für jedes Element, dass nicht „0“ ist, eine neue Instanz unserer Funktion. Diese neue Funktion kriegt jeweils veränderte Daten:
  - Einen Häufigkeitsvektor, bei dem die betroffene Figur um eins reduziert wurde.
  - Einen neuen Vektor der ausgeführten Figuren, der nun um eins länger ist und die ausgewählte Figur als letztes Element beinhaltet.
  - Neue Positionen der Tänzer. Mit der gespeicherten ‚reconfig‘ der ausgewählten Figur ändern wir die Positionen der Tänzer.

Wir wiederholen diese rekursive Funktion für jede mögliche Häufigkeitsverteilung und zeichnen alle passenden Choreographien in einem Vektor auf.

Wenn es schlussendlich mehrere mögliche Choreographien gibt, dann müssen wir sie nach den aufgabenbezogenen Kriterien bewerten. Alle wichtige Information für eine Bewertung einer Choreographie speichern wir in folgender Figur (Q1) :

```
leaderboard_rating :: struct {  
    idx : int,  
    different_figures : int,  
    num_of_figures : int,  
    total_distance : int  
}
```

Wir erstellen einen weiteren Vektor von solchen „leaderboard\_rating“. Da wir diesen häufig umsortieren, speichern wir in dem Feld „idx“, die eigentliche Position der dazugehörigen Choreographie im Choreographievektor. Außerdem ermitteln wir hier die drei Parameter, die wir benötigen um die verschiedenen Choreographien zu bewerten (Q5) :

- „different\_figures“: Wir erstellen uns für jede Choreographie einen Vektor von Boolean „false“, mit der gleichen Länge wie die Anzahl von Figuren. Ein Element an Stelle i in diesem Vektor, bedeutet, ob die Figur an der gleichen Stelle in der dazugehörigen Choreographie vorkommt. Hierfür gehen wir durch jede Figur einer Choreographie durch und setzen die gleiche Choreographie in unserem Boolean Vektor auf „true“. Wir müssen nur dann für eine spezifische Choreographie zählen, wie viele Elemente in diesem Vektor „true“ sind, um zu wissen wie viele verschiedene Figuren dort auftauchen. Wir speichern daraufhin diese Information in der dazugehörigen „leaderboard\_rating“ Struktur.

- „num\_of\_figures“: Für jede Choreographie können wir das „num\_of\_figures“ Feld in ihrer „leaderboard\_rating“ Struktur einfach mit der Länge ihres Choreographievektors füllen.
- „total\_distance“: Wir berechnen zuvor die Distanz, die bei allen Figuren zurückgelegt wird. Dafür schauen wir uns einfach deren „reconfig“ an und summieren die Distanz zwischen jedes Tänzers Ausgangs und Endposition. Wir gehen dann durch jede Figur der Choreographie durch und summieren dessen zugehörige Distanz. Der Endwert kann nun in die jeweilige „leaderboard\_rating“ Struktur gefüllt werden.

Nun sortieren wir wiederholte male den „leaderboard\_rating“ Vektor nach den verschiedenen Kriterien und suchen uns jeweils die meist passende Choreographie aus, die dann als Ausgabe zurückgegeben werden.

## Werkzeuge

### Packages:

core:fmt – Die Ergebnisse ins Terminal zu schreiben, Fehler beim Lesen und Schreiben von Dateien zurückzugeben und Integer als strings zu formatieren.

core:unicode/utf8 – Eingaben strings in eine Liste von Zeichen umwandeln für einfacheren Umgang.

core:os – Schreiben und lesen von Eingaben und Ausgabendateien.

core:strings – Verknüpfen von Ausgaben und Dateipfadstrings, teilen und umwandeln von strings zu cstrings, da raylib cstrings benötigt.

core:sort – Sortieren von Listen um bei der Präsentation der drehfreudigen Bäume sie in der richtigen Reihenfolge zu haben.

core:math – Um die Abrundung aus **Gleichung 1** zu implementieren.

core:path/filepath – Einfacherer und plattformübergreifender Umgang mit Dateieinpfaden.

core:strconv – Inputstrings in Integer umwandeln.

## Beispiele

| Beispiel     | Ausgabe                                                                                                                                                                                                                                                                         |
|--------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| choreo01.txt | Die Choreographie mit möglichst vielen unterschiedlichen Figuren lautet: Cast_Four, Cast_Up, Cast_Four, Cast_Up, Cast_Four, Cast_Up, Cast_Four, Cast_Up, Cast_Four, Cast_Four, Cast_Four. Und hat 2 verschiedene Figuren. Es gibt außerdem 11 Choreographien, die genauso viele |



|              |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|--------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|              | <p>unterschiedliche Figuren haben.</p> <p>Die Choreographie mit den meisten Figuren lautet: Cast_Four, Cast_Four, Cast_Four, Cast_Four, Cast_Four, Cast_Four, Cast_Four, Cast_Four, Cast_Four, Cast_Four, Cast_Four, Cast_Four, Cast_Four, Cast_Four, Cast_Four. Und hat 16 Figuren. Es gibt außerdem 0 Choreographien, die genauso viele Figuren haben.</p> <p>Die Choreographie mit den wenigsten Figuren lautet: Cast_Up, Cast_Four, Cast_Up, Cast_Four, Cast_Up, Cast_Four, Cast_Four, Cast_Four, Cast_Four, Cast_Four, Cast_Four, Cast_Up, Cast_Four. Und hat 12 Figuren. Es gibt außerdem 11 Choreographien, die genauso wenige Figuren haben.</p> <p>Die Choreographie mit der größt möglich zurückgelegten Strecke lautet: Cast_Four, Cast_Four, Cast_Four, Cast_Four, Cast_Four, Cast_Four, Cast_Four, Cast_Four, Cast_Four, Cast_Four, Cast_Four, Cast_Four, Cast_Four, Cast_Four, Cast_Four. Und es wird eine Distanz von 384 zurückgelegt. Es gibt außerdem 0 Choreographien, die auch die gleiche zurückgelegte Strecke haben.</p> <p>Die Choreographie mit der kleinst möglich zurückgelegten Strecke lautet: Cast_Four, Cast_Up, Cast_Four, Cast_Up, Cast_Four, Cast_Four, Cast_Four, Cast_Four, Cast_Four, Cast_Up, Cast_Four, Cast_Up. Und es wird eine Distanz von 312 zurückgelegt. Es gibt außerdem 11 Choreographien, die auch die gleiche zurückgelegte Strecke haben.</p> |
| choreo02.txt | Keine Choreographien gefunden.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| choreo03.txt | Keine Choreographien gefunden.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| choreo04.txt | <p>Die Choreographie mit möglichst vielen unterschiedlichen Figuren lautet: Aida, Fan, Fan, Hover, Fan, Hockey_Stick, Fan, Fan, Fan, Fan, Appel, Bounce. Und hat 6 verschiedene Figuren. Es gibt außerdem 11 Choreographien, die genauso viele unterschiedliche Figuren haben.</p> <p>Die Choreographie mit den meisten Figuren lautet: Fan, Fan, Hover, Fan, Hockey_Stick, Fan, Fan, Fan, Fan, Appel, Bounce, Aida. Und hat 12 Figuren. Es gibt außerdem 11 Choreographien, die genauso viele Figuren haben.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |

|              |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|--------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|              | <p>Die Choreographie mit den wenigsten Figuren lautet: Hover, Fan, Hockey_Stick, Fan, Fan, Fan, Fan, Appel, Bounce, Aida, Fan, Fan. Und hat 12 Figuren. Es gibt außerdem 11 Choreographien, die genauso wenige Figuren haben.</p> <p>Die Choreographie mit der größt möglich zurückgelegten Strecke lautet: Fan, Appel, Bounce, Aida, Fan, Fan, Hover, Fan, Hockey_Stick, Fan, Fan, Fan. Und es wird eine Distanz von 1154 zurückgelegt. Es gibt außerdem 11 Choreographien, die auch die gleiche zurückgelegte Strecke haben.</p> <p>Die Choreographie mit der kleinst möglich zurückgelegten Strecke lautet: Fan, Hover, Fan, Hockey_Stick, Fan, Fan, Fan, Fan, Appel, Bounce, Aida, Fan. Und es wird eine Distanz von 1154 zurückgelegt. Es gibt außerdem 11 Choreographien, die auch die gleiche zurückgelegte Strecke haben.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| choreo05.txt | <p>Die Choreographie mit möglichst vielen unterschiedlichen Figuren lautet: Hip_Twist, Hip_Twist, Spot_Turn, Spot_Turn, Spot_Turn, Hip_Twist, Hip_Twist, Hip_Twist, Hip_Twist, Hip_Twist, Hip_Twist, Spot_Turn. Und hat 2 verschiedene Figuren. Es gibt außerdem 1249 Choreographien, die genauso viele unterschiedliche Figuren haben.</p> <p>Die Choreographie mit den meisten Figuren lautet: Hip_Twist, Hip_Twist, Hip_Twist, Hip_Twist, Hip_Twist, Hip_Twist, Hip_Twist, Hip_Twist, Hip_Twist, Hip_Twist, Hip_Twist, Hip_Twist, Hip_Twist, Hip_Twist, Hip_Twist, Hip_Twist, Hip_Twist, Hip_Twist, Hip_Twist, Hip_Twist, Hip_Twist, Hip_Twist, Hip_Twist, Hip_Twist. Und hat 24 Figuren. Es gibt außerdem 0 Choreographien, die genauso viele Figuren haben.</p> <p>Die Choreographie mit den wenigsten Figuren lautet: Spot_Turn, Spot_Turn, Spot_Turn, Spot_Turn, Spot_Turn, Spot_Turn. Und hat 6 Figuren. Es gibt außerdem 0 Choreographien, die genauso wenige Figuren haben.</p> <p>Die Choreographie mit der größt möglich zurückgelegten Strecke lautet: Hip_Twist, Hip_Twist, Hip_Twist, Hip_Twist, Hip_Twist, Hip_Twist, Hip_Twist, Hip_Twist, Hip_Twist, Hip_Twist. Und hat 10 Figuren. Es gibt außerdem 0 Choreographien, die genauso viele Figuren haben.</p> |

|              |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|--------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|              | <p>Hip_Twist, Hip_Twist, Hip_Twist, Hip_Twist, Hip_Twist, Hip_Twist, Hip_Twist, Hip_Twist, Hip_Twist, Hip_Twist, Hip_Twist, Hip_Twist, Hip_Twist, Hip_Twist, Hip_Twist. Und es wird eine Distanz von 1728 zurückgelegt. Es gibt außerdem 0 Choreographien, die auch die gleiche zurückgelegte Strecke haben.</p> <p>Die Choreographie mit der kleinst möglich zurückgelegten Strecke lautet: Spot_Turn, Spot_Turn, Spot_Turn, Spot_Turn, Spot_Turn, Spot_Turn. Und es wird eine Distanz von 504 zurückgelegt. Es gibt außerdem 0 Choreographien, die auch die gleiche zurückgelegte Strecke haben.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| choreo06.txt | <p>Die Choreographie mit möglichst vielen unterschiedlichen Figuren lautet: Cycle_4, Cycle_Back, Cycle_4, Sides, Pairs, Cycle_4, Pairs, Cycle_Back, Cycle_Back, Cycle_Back. Und hat 4 verschiedene Figuren. Es gibt außerdem 10801 Choreographien, die genauso viele unterschiedliche Figuren haben.</p> <p>Die Choreographie mit den meisten Figuren lautet: Cycle_Back, Pairs, Cycle_Back, Pairs, Cycle_Back, Pairs, Cycle_Back, Pairs, Cycle_Back, Pairs, Cycle_Back, Pairs, Cycle_Back, Pairs, Cycle_Back, Pairs. Und hat 16 Figuren. Es gibt außerdem 3 Choreographien, die genauso viele Figuren haben.</p> <p>Die Choreographie mit den wenigsten Figuren lautet: Reverse, Sides, Reverse, Sides. Und hat 4 Figuren. Es gibt außerdem 7 Choreographien, die genauso wenige Figuren haben.</p> <p>Die Choreographie mit der größt möglich zurückgelegten Strecke lautet: Cycle_4, Cycle_4, Cycle_4, Cycle_4, Cycle_4, Cycle_4, Cycle_4. Und es wird eine Distanz von 768 zurückgelegt. Es gibt außerdem 0 Choreographien, die auch die gleiche zurückgelegte Strecke haben.</p> <p>Die Choreographie mit der kleinst möglich zurückgelegten Strecke lautet: Pairs, Pairs, Pairs, Pairs, Pairs, Pairs, Pairs, Pairs, Pairs, Pairs, Pairs, Pairs. Und es wird eine Distanz von 256 zurückgelegt. Es gibt außerdem 0 Choreographien, die auch die gleiche zurückgelegte Strecke haben.</p> |

|             |                                |
|-------------|--------------------------------|
| choreoA.txt | Keine Choreographien gefunden. |
|-------------|--------------------------------|

## Quellcode

### Grundstrukturen erstellen (Q1):

```

Configuration :: [16]int
base_config : Configuration : {0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15}

Action :: struct {
    name : string,
    length : int,
    reconfig : Configuration
}

leaderboard_rating :: struct {
    idx : int,
    different_figures : int,
    num_of_figures : int,
    total_distance : int
}

```

### Figuren einlesen und sortieren (Q2):

```

for i in 2..<len(parsed_input) {
    if len(parsed_input[i]) == 0 {
        actions = actions[0:(len(actions) - 1)]
        continue
    }

    act : Action
    splitted := strings.split(parsed_input[i], " ")
    act.name = splitted[0]
    act.length = strconv.Atoi(splitted[1])
    runeArray := utf8.String_to_Runes(splitted[2])
    for i2 in 0..<len(runeArray) {
        act.reconfig[i2] = letter_map[runeArray[i2]]
    }
    actions[i-2] = act
}

```

### Häufigkeitsverteilungen finden (Q3):

```

action_exploration :: proc(actions : []Action, action_bounds : []int,
total_length : int, combis : ^[dynamic][]int) {

    action_testable_combis := make([dynamic][]int)
    action_testable_combis_lengths := make([dynamic]int)

    for i in 0..=action_bounds[0] {
        temp_holder := make([]int, 1)
        temp_holder[0] = i

        append(&action_testable_combis, temp_holder)
        append(&action_testable_combis_lengths, i * actions[0].length)
    }
}

```

```

    }
    for len(action_testable_combis) > 0 {
        action_length := action_testable_combis_lengths[0]
        if action_length > total_length {
            delete(action_testable_combis[0])
            unordered_remove(&action_testable_combis, 0)
            unordered_remove(&action_testable_combis_lengths, 0)
            continue
        }
        if len(action_testable_combis[0]) == len(actions) {
            if action_length == total_length {
                append(combis, action_testable_combis[0])
            } else {
                delete(action_testable_combis[0])
            }
        } else {
            new_action_index := len(action_testable_combis[0])
            for i in 0..action_bounds[new_action_index] {
                temp_actions := make([]int,
len(action_testable_combis[0]) + 1)
                copy(temp_actions, action_testable_combis[0])

                temp_actions[new_action_index] = i
                append(&action_testable_combis, temp_actions)

                new_action_length := actions[new_action_index].length *
i
                append(&action_testable_combis_lengths, action_length +
new_action_length)
            }

            delete(action_testable_combis[0])
        }
        unordered_remove(&action_testable_combis, 0)
        unordered_remove(&action_testable_combis_lengths, 0)
    }
    delete(action_testable_combis)
}

```

#### Rekursives funktionierende Permutationen finden (Q4):

```

recursive_combi_finding :: proc(actions : []Action, actions_left : []int ,
actions_done : []int, config : Configuration) -> [dynamic][]int {
    empty := true
    for left in actions_left {
        if left > 0 {empty = false}
    }

    if empty {
        if config != base_config {
            delete(actions_done)

```

```

        delete(actions_left)
        return make([dynamic][]int)
    }

    delete(actions_left)
    temp := make([dynamic][]int)
    append(&temp, actions_done)
    return temp
} else {

    findings := make([dynamic][]int)

    for i in 0..<len(actions) {
        if actions_left[i] == 0 {continue}

        temp_actions_left := make([]int, len(actions_left))
        copy(temp_actions_left, actions_left)
        temp_actions_left[i] -= 1

        temp_actions_done := make([]int, len(actions_done) + 1)
        copy(temp_actions_done, actions_done)
        temp_actions_done[len(actions_done)] = i

        temp_config := reconfigure(config, actions[i].reconfig)

        finding := recursive_combi_finding(actions, temp_actions_left,
            temp_actions_done, temp_config)
        for find in finding {
            append(&findings, find)
        }
    }

    delete(actions_left)
    delete(actions_done)
    return findings
}
}

```

**Passende Choreographien bewerten (Q5):**

```

ratings := make([]leaderboard_rating, len(ta_comb))
defer delete(ratings)

action_distances := find_actions_total_distance(actions)

for i in 0..<len(ta_comb) {
    ratings[i].idx = i
    ratings[i].num_of_figures = len(ta_comb[i])
    actions_existing := make([]bool, len(actions))
    for act in ta_comb[i] {
        actions_existing[act] = true
    }
    unique_actions : int
    for exists in actions_existing {
        if exists {unique_actions += 1}
    }
    ratings[i].different_figures = unique_actions
    total_distance : int

```

```
    for act in ta_comb[i] {  
        total_distance += action_distances[act]  
    }  
    ratings[i].total_distance = total_distance  
}
```